A FRIENDLY WALKTHROUGH FOR A NEW TEAMMATE

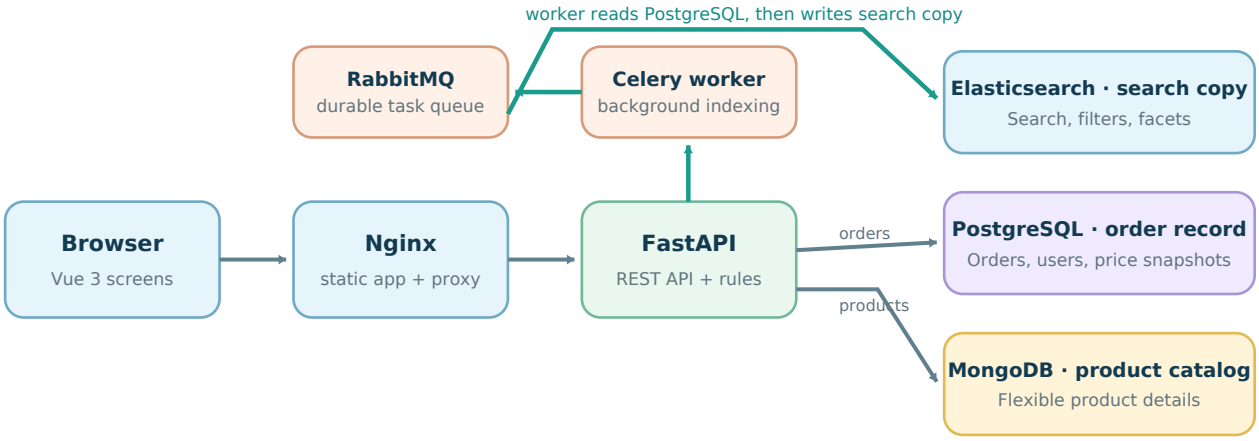
BuyMore Project Architecture

Knowledge Bytes: small explanations of what each part does, how data moves, and why the system is arranged this way.

Based on the repository as it exists today · Vue 3 · FastAPI · PostgreSQL · MongoDB · Elasticsearch · RabbitMQ · Celery

Start with the whole picture

BuyMore is a small online shop and order-management demo. A browser app talks to a Python API. The API uses different stores for products, trustworthy order records, and fast order search. A background worker keeps the search copy updated after order changes.



Reading path Browser → Nginx → FastAPI → correct data store. Product browsing reads MongoDB; order details read PostgreSQL; order search reads Elasticsearch.

Order update path FastAPI → commit PostgreSQL → RabbitMQ task → Celery → Elasticsearch. This is asynchronous: an order can be saved before it appears in search.

Byte index

04 · API boundaries and validation
05 · Why three data stores
06 · Product c
12 · Frontend state and price snapshots
13 · Startup, seed and containers
14 · Failure c

BYTE 01

What BuyMore is for

Builds on: None — starting point

BuyMore is a learning project that combines a customer storefront with simple admin tools. Its main architecture lesson is that the product catalog, official order records, and fast search serve different jobs and are stored separately.

```
frontend/             customer + admin screens
backend/app/          API and data rules
docker-compose.yml    local services and wiring
```

Why it matters: The project is a proof of concept. It demonstrates a queued search-indexing flow; it is not a production checkout system with authentication, payments, or shipping integrations.

BYTE 02

The five screens are one app

Builds on: Byte 01

Vue Router selects one of five views inside a shared layout: shopping, cart checkout, order details, order search, or catalog management. The shared header keeps the cart and shopper selector available across pages.

```
{ path: '/', component: Shop },
{ path: '/checkout', component: Checkout },
{ path: '/admin/search', component: Search },
{ path: '/admin/orders/:id', component: OrderDetails },
{ path: '/admin/catalog', component: Catalog }
```

Why it matters: The screens are frontend routes, not separate server applications. They all use the same API and shared reactive cart state.

BYTE 03

How a browser request gets to the API

Builds on: Byte 02

The browser calls paths beginning with /api. In the container setup, Nginx serves the built Vue files and forwards API paths to FastAPI. A same-origin path keeps frontend requests simple.

```
// frontend/src/api.js
fetch('/api' + path, {
  ...options,
  headers: {'Content-Type': 'application/json', ...options.headers}
})
```

Why it matters: Nginx also returns index.html for frontend routes, so refreshing a URL such as /admin/search still loads the single-page app.

BYTE 04

FastAPI is the rule and routing layer

Builds on: Byte 03

FastAPI exposes endpoints for users, products, orders, status updates, and search. Pydantic schemas check incoming JSON before route logic runs; the route then applies business rules and calls the relevant store.

```
@app.post('/api/orders', status_code=201)
def place_order(payload: OrderCreate):
    ...
```

Why it matters: For example, schemas limit cart quantities and require valid product IDs; route logic additionally verifies that the chosen shopper exists and each product remains active.

BYTE 05

Three stores, three responsibilities

Builds on: Byte 04

MongoDB holds the flexible catalog. PostgreSQL is the trusted record for orders and money. Elasticsearch is a derived copy optimized for searching and dashboard summaries.

```
MongoDB       mutable products
PostgreSQL    users + orders + purchased-item snapshots
Elasticsearch indexed order documents for search
```

Why it matters: A derived copy means it can be rebuilt from the authoritative record. Search does not silently fall back to another database if Elasticsearch is offline; it reports that search is unavailable.

BYTE 06

Catalog products live in MongoDB

Builds on: Byte 05

Products have fields that vary by type, such as color, technical attributes, and variants. MongoDB documents can store those fields without forcing every product into the same fixed set of columns.

```
# backend/app/main.py
criteria = {'active': True}
if category:
    criteria['category'] = category
items = products.find(criteria).sort('sku', 1) \
    .skip((page - 1) * page_size).limit(page_size)
```

Why it matters: The API supports search, category and price filters, inactive products for catalog management, and pagination. A unique SKU index rejects duplicate product codes.

BYTE 07

PostgreSQL protects the order record

Builds on: Byte 05

An order, its customer, and its purchased lines are relational data: one order belongs to a shopper and contains one or more line items. PostgreSQL transactions keep the order and its lines together.

```
class Order(Base):
    __tablename__ = 'orders'
    user_id: Mapped[int] = mapped_column(ForeignKey('users.id'))
    total_amount: Mapped[Decimal] = mapped_column(Numeric(12, 2))
    version: Mapped[int] = mapped_column(Integer, default=1)
```

Why it matters: Database constraints also reject impossible states such as a negative order total or a quantity of zero. Decimal and fixed-scale numeric columns avoid ordinary floating-point price arithmetic.

BYTE 08

Checkout turns a cart into a saved order

Builds on: Bytes 04, 06, 07

The cart is sent as product IDs and quantities. The API checks the current catalog and recalculates every line price on the server, so the browser's displayed price is not trusted as the final amount.

```
with Session() as db:
    with db.begin():
        lines = [OrderItem(
            product_id=pid,
            title=catalog[pid]['title'],
            quantity=qty,
            unit_price=Decimal(str(catalog[pid]['price']))
        ) for pid, qty in quantities.items()]
        order = Order(items=lines, total_amount=...)
        db.add(order)
```

Why it matters: The transaction commits the order and items as one unit. If a line insert fails, the order insert rolls back too. Only after the transaction does the API try to queue search indexing.

BYTE 09

RabbitMQ and Celery move slow work out of checkout

Builds on: Byte 08

RabbitMQ is a mailbox for background tasks. Celery is the worker framework that takes tasks from that mailbox and runs them. Checkout can finish after the database commit without waiting for a search index update.

```
# backend/app/sync.py
@celery.task(autoretry_for=(Exception,),
             retry_backoff=2, max_retries=20)
def sync_order(order_id):
    index_order(order_id)
```

Why it matters: The worker re-reads the latest committed order from PostgreSQL before creating its search document. This prevents a delayed task from blindly carrying an old payload.

BYTE 10

Elasticsearch powers order search

Builds on: Bytes 05, 09

The API copies an order into a searchable document with customer and item details. Elasticsearch can then match customer names, emails, and product titles while also applying status, date, and value filters.

```
must = [{'multi_match': {
  'query': q.query.strip(),
  'fields': ['customer.name^2',
             'customer.email', 'items.title'],
  'type': 'bool_prefix'
}}]
```

Why it matters: Search, result pages, status counts, and revenue use the same filtered search query. A partial final word such as “Wirel” can match “Wireless”; arbitrary middle-of-word substring matching is not promised.

BYTE 11

Status changes create a newer version

Builds on: Bytes 07, 09, 10

When staff change an order status, the API locks and updates the PostgreSQL order, updates its timestamp, and increments a version. It then queues a task to refresh the search copy.

```
order.status = payload.status
order.updated_at = utcnow()
order.version += 1
# after commit:
dispatch(order_id)
```

Why it matters: Elasticsearch uses external versioning. A delayed older task cannot overwrite a newer indexed version. The order details screen compares SQL and search versions to show whether the search copy is current.

BYTE 12

The browser owns the temporary cart; orders keep snapshots

Builds on: Bytes 06, 08

The cart is reactive Vue state and is saved in browser local storage. It is a convenient draft, not the official order. At checkout the server copies product title and price into each SQL order item.

```
watch(() => [store.userId, store.cart], () =>
  localStorage.setItem('buymore-cart',
    JSON.stringify({userId: store.userId, cart: store.cart})),
  {deep: true})
```

Why it matters: Catalog details can change later, but an old receipt must still describe what was bought at the time. That is why the order item stores a title and unit-price snapshot instead of relying on the current product document.

BYTE 13

Compose starts the system in dependency order

Builds on: Bytes 03–12

Docker Compose runs the databases, broker, setup job, API, worker, and frontend on a private application network. Health checks and dependency rules make setup wait until its data services are ready.

```
setup: migrate + seed
api, worker: wait for setup to finish
frontend: serves UI and proxies /api
```

Why it matters: The one-time setup creates the PostgreSQL schema, Mongo SKU index, and Elasticsearch mapping, then loads sample data and reconciles orders. Named volumes preserve database files when containers stop.

BYTE 14

What happens when a dependency fails

Builds on: Bytes 08–13

If Elasticsearch is down, SQL checkout can still save an order. The queue and worker retry indexing, while search returns a clear service-unavailable response instead of a fake empty result.

```
dispatch() → 'queued' or 'queue_unavailable'
sync_state() → in_sync / pending / unavailable
python -m scripts.reindex_orders
```

Why it matters: There is a small gap between committing SQL and publishing to RabbitMQ. These are separate actions, so the system is not exactly-once. If publication fails, the order remains safe in PostgreSQL and the reindex script can rebuild the search copy.

BYTE 15

A good path through the code

Builds on: All earlier bytes

Follow one user action across the layers to learn the project. For checkout, start in the Vue view, find the API helper, inspect the FastAPI route and model, then trace the queued task into the indexer.

```
frontend/src/views/Checkout.vue
→ frontend/src/api.js
→ backend/app/main.py : place_order
→ backend/app/models.py : Order, OrderItem
→ backend/app/sync.py : dispatch, sync_order, index_order
→ backend/app/search.py : mapping + query
```

Why it matters: For runtime wiring, read docker-compose.yml. For validation examples and failure behavior, read backend/tests/test_project.py. The root README gives the quick setup and endpoint overview.

ONE LAST MENTAL MODEL

Three questions to ask about any data

Is this the current product? → MongoDB is the catalog.

Is this what the customer actually ordered? → PostgreSQL is the record.

Do I need to find matching orders quickly? → Elasticsearch is the search copy.

If the search copy is stale, the worker catches up from the PostgreSQL record. If automatic delivery was missed, reconciliation can rebuild it.

Prepared from the BuyMore repository source. This guide describes the code’s current local proof-of-concept architecture and its intentional limitations.